

DuckDB Internals and CO₂ Analytics Application

Final Report — DSCI 551: Foundations of Data Management

Junwei Yin | USC ID: 8746721183 | junweiyi@usc.edu

University of Southern California — Spring 2026

1. Introduction & Motivation

Modern data analysis increasingly relies on databases that are optimized for analytical workloads rather than traditional transactional systems. This project explores DuckDB, an in-process analytical database, through the lens of a real-world CO₂ emissions analytics application.

The primary motivation for choosing DuckDB is its design philosophy: instead of requiring a separate server process, DuckDB runs embedded within the application itself, making it lightweight and easy to integrate with Python-based data tools. More importantly, DuckDB exposes its internal optimizations in a way that is directly observable, making it an ideal system for studying how database internals affect application performance.

The application we built, a CO₂ per capita analytics dashboard, is not simply a visualization tool. Each feature is deliberately designed to trigger a specific internal DuckDB mechanism — serving as a performance probe. The dataset used is the CO₂ emissions per capita dataset from Our World in Data, covering 26,508 records across 200+ countries from 1949 to 2024.

The two core internal focus areas of this project are: **(1) query execution optimization**, specifically the TOP-N heap sort operator, and **(2) storage-layer scan optimization**, specifically predicate pushdown and projection pushdown enabled by the Parquet file format.

2. Database System Overview

DuckDB is an open-source, in-process analytical database management system designed for Online Analytical Processing (OLAP) workloads. It was developed at CWI Amsterdam and first publicly released in 2019. Unlike traditional client-server databases such as MySQL or PostgreSQL, DuckDB runs entirely within the host application process, requiring no separate server setup or network communication.

DuckDB's primary design goals are:

- **In-process execution:** DuckDB runs inside the same process as the application, eliminating network overhead.
- **Columnar storage:** Data is stored and processed column by column, enabling efficient analytical queries.
- **Vectorized execution:** Queries are processed in batches of column values rather than row by row, improving CPU cache efficiency.
- **Zero-copy integration:** DuckDB integrates with Python's Pandas and Apache Arrow without serialization overhead.

DuckDB is particularly well-suited for analytical workloads such as filtering large datasets, aggregating metrics across time periods, and ranking records. These are precisely the operations performed in our CO₂ analytics application.

3. Internal Architecture

3.1 Query Execution Pipeline

When a SQL query is submitted to DuckDB, it passes through four stages: parsing, logical planning, physical optimization, and vectorized execution. The physical optimizer is responsible for selecting the most efficient execution operators for a given query.

A key optimization relevant to this project is the TOP-N operator. When a query contains both ORDER BY and LIMIT clauses, DuckDB's optimizer recognizes this pattern and replaces the standard sort operator with a heap-based TOP-N scan. Instead of materializing a full sorted list of all rows ($O(N \log N)$), the TOP-N operator maintains a small heap of size K during the scan ($O(N \log K)$). Each incoming row is compared against the current minimum of the heap; rows that do not qualify are immediately discarded. This reduces both computational cost and memory usage.

A second execution optimization is the PERFECT_HASH_GROUP_BY operator. When a GROUP BY clause produces a small, predictable number of groups — such as grouping by Year, which yields only a few dozen distinct values — DuckDB automatically selects a perfect hash function. Unlike a general hash table, a perfect hash assigns each group key a unique, collision-free slot, eliminating the overhead of collision resolution.

3.2 Storage Layer and Scan Optimization

DuckDB's storage architecture supports multiple file formats, with Parquet being the most optimized for analytical queries. Parquet is a columnar binary format that stores data in row groups, each accompanied by statistical metadata (Zone Maps) recording the minimum and maximum values of each column within that group.

This metadata enables two critical scan-time optimizations:

- **Predicate Pushdown:** When a WHERE clause filter is applied, DuckDB pushes the predicate into the PARQUET_SCAN operator. Row groups whose Zone Map statistics guarantee no matching values are skipped entirely, meaning the filtered data never enters memory.
- **Projection Pushdown:** DuckDB reads only the columns required by the query. Since Parquet stores each column independently, unused columns are never read from disk.

In contrast, CSV is a row-oriented text format with no statistical metadata. DuckDB must scan every row and every column of a CSV file before applying any filter, making it significantly less efficient for selective analytical queries.

3.3 Vectorized Execution

DuckDB processes data in batches of column values rather than one row at a time. Each batch (called a vector) typically contains 1,024 values of a single column. This approach is cache-friendly because contiguous column values fit efficiently into CPU cache lines, reducing memory latency. Vectorized execution is applied across all

operators, including scans, filters, aggregations, and joins.

4. Application Design

4.1 Overview

The application is a CO₂ emissions analytics dashboard built using Streamlit, a Python-based web application framework. DuckDB is integrated directly into the Python process, allowing SQL queries to be executed against local data files without any server setup. The application is designed around the concept of a performance probe: each feature is constructed to trigger a specific DuckDB internal mechanism, making the internal behavior observable through EXPLAIN plan output.

4.2 Dataset

The dataset is the CO₂ emissions per capita dataset sourced from Our World in Data. It contains 26,508 records covering over 200 countries and regions from 1949 to 2024, with four columns: Entity (country name), Code (ISO country code), Year, and CO₂ emissions per capita (measured in tonnes per person). The dataset was originally downloaded in CSV format and converted to Parquet using a preprocessing script to enable Parquet-based scan optimizations.

4.3 Application Features

The application consists of four features, each corresponding to a specific internal mechanism:

- **Top Emitters:** Users select a year and a K value to retrieve the top K highest-emitting countries. This triggers the TOP-N operator and Parquet predicate pushdown.
- **Country Comparison:** Users select multiple countries and a year range to compare CO₂ trends over time. This triggers predicate pushdown via an IN-clause filter.
- **CSV vs. Parquet Benchmark:** The same Top-K query is executed on both CSV and Parquet formats and timed across multiple runs. This directly demonstrates the performance impact of predicate pushdown.
- **Trend Explorer:** Users run GROUP BY aggregations over a selected year range. This triggers the PERFECT_HASH_GROUP_BY operator and projection pushdown.

Each feature displays the query result alongside the physical EXPLAIN plan, allowing the user to directly observe which internal operators DuckDB selected for that query.

5. Mapping Internals to Application Behavior

5.1 Top Emitters → TOP-N Operator + Predicate Pushdown

When the user selects Year = 2022 and K = 10, the application executes:

```
SELECT Entity, Code, Year, "CO2 emissions per capita" AS co2_pc FROM 'data/co-emissions-per-capita.parquet' WHERE Year = 2022 ORDER BY co2_pc DESC LIMIT 10
```

The EXPLAIN plan reveals two internal mechanisms working in sequence. First, the PARQUET_SCAN node applies Filters: Year=2022 directly within the scanner, reducing 26,508 rows to approximately 5,301 rows before any other processing occurs. This is predicate pushdown — the filter is evaluated at the storage layer rather than after a full scan.

Second, the TOP-N operator receives these 5,301 rows and maintains a heap of size 10 during the scan. Each row is compared against the current minimum of the heap; rows that do not qualify are discarded immediately. The result is that memory never holds more than 10 rows at any point, regardless of how many rows pass the predicate filter. The algorithmic complexity drops from $O(N \log N)$ for a full sort to $O(N \log K)$ for TOP-N, a significant improvement when K is small.

Without predicate pushdown, the TOP-N operator would receive all 26,508 rows. Without TOP-N, the system would sort all 5,301 filtered rows before discarding 5,291 of them. The combination of both optimizations means **DuckDB processes the minimum possible data at every stage of the pipeline.**

5.2 CSV vs. Parquet Benchmark → Predicate Pushdown Impact

The benchmark feature runs the same Top-K query on both CSV and Parquet formats and measures the average execution time across multiple runs. The results consistently show Parquet outperforming CSV, with observed speedups ranging from 3x to 10x depending on the selected year.

Examining the EXPLAIN plans side by side reveals the root cause. In the CSV plan, DuckDB uses the READ_CSV_AUTO operator at the scan level, with the Year filter appearing as a separate FILTER node above the scan. This means all rows are read and parsed from the CSV file before any filtering occurs. In the Parquet plan, the Year filter appears inside the PARQUET_SCAN node itself, with row counts reduced at the scan level.

The performance difference is not due to a different ranking algorithm — both plans use TOP-N. The difference is entirely in how much data reaches the TOP-N operator. In CSV, 26,508 rows enter the pipeline. In Parquet, only the rows matching the year filter enter the pipeline. Scan cost dominates this query, and predicate pushdown eliminates the majority of that cost before execution begins.

5.3 Country Comparison → Predicate Pushdown via IN-Clause

When the user selects multiple countries, the application generates a WHERE Entity IN (...) clause. The EXPLAIN plan shows that DuckDB pushes this filter into the PARQUET_SCAN node as an optional predicate, narrowing the initial scan from 26,508 rows to approximately 5,301 rows. A subsequent FILTER node performs an exact row-level verification using OR conditions to confirm the final result set.

This two-stage filtering pattern — coarse pushdown at the scan level followed by precise row-level filtering — demonstrates that predicate pushdown is not limited to

equality filters on numeric columns. DuckDB applies the same optimization to string-based IN-clause filters, using Parquet's Zone Map statistics to skip row groups that cannot contain any of the specified entity names.

5.4 Trend Explorer → PERFECT_HASH_GROUP_BY + Projection Pushdown

The Trend Explorer executes a GROUP BY Year aggregation over a selected year range. The EXPLAIN plan shows two notable internal behaviors. At the scan level, the PARQUET_SCAN node reads only the Year and CO₂ columns, skipping the Entity and Code columns entirely. This is projection pushdown — DuckDB reads only the columns required by the query.

For the aggregation itself, DuckDB selects the PERFECT_HASH_GROUP_BY operator rather than a general hash-based GROUP BY. Because Year produces only a small, bounded number of distinct groups (at most a few dozen years in any given range), DuckDB's optimizer detects that a perfect hash function can be constructed. A perfect hash assigns each group key a unique, collision-free slot, eliminating the overhead of collision resolution that a general hash table requires. The result is a faster aggregation with lower memory overhead.

6. Comparison with MySQL and MongoDB

6.1 DuckDB vs. MySQL

MySQL is a row-oriented relational database designed primarily for Online Transaction Processing (OLTP) workloads — operations such as single-row inserts, point lookups, and short transactions. DuckDB is designed for the opposite workload: large-scale analytical scans, aggregations, and rankings across many rows.

The fundamental architectural difference is storage layout. MySQL stores data row by row, meaning that reading any column requires reading the entire row. For a query that only needs the CO₂ column out of four, MySQL reads four times more data than necessary. DuckDB's columnar storage reads only the requested columns.

MySQL also lacks native support for Parquet files and the associated predicate pushdown optimizations. A comparable analytical query in MySQL would require loading the entire dataset into a table, with no ability to skip row groups at the file level. DuckDB's integration with Parquet's Zone Map statistics enables skip-level filtering that MySQL cannot replicate without custom indexing.

That said, MySQL is superior for workloads involving frequent writes, complex multi-table transactions, and row-level locking. For the CO₂ analytics use case, which is purely read-only and analytical, DuckDB is the more appropriate choice.

Dimension	DuckDB	MySQL
Storage model	Columnar	Row-oriented
Primary workload	OLAP (analytical)	OLTP (transactional)

Predicate pushdown	Yes (Parquet)	No native support
Deployment	In-process, no server	Client-server
Write performance	Limited	Optimized
Best use case	Analytics, reporting	Web apps, transactions

6.2 DuckDB vs. MongoDB

MongoDB is a document-oriented NoSQL database that stores data as JSON-like BSON documents. It is designed for flexible schema workloads where data structure may vary across records and where horizontal scaling across multiple nodes is required.

For the CO₂ analytics use case, MongoDB would be a poor fit. Analytical aggregations such as GROUP BY Year or ORDER BY CO₂ DESC LIMIT 10 require MongoDB's aggregation pipeline, which is more verbose and generally less performant than DuckDB's SQL optimizer for columnar data. MongoDB stores documents as rows, so the same projection pushdown and column-skipping optimizations available in DuckDB are not natively applicable.

MongoDB's strengths lie in flexible document structures, horizontal sharding across distributed nodes, and workloads where data relationships are hierarchical or nested. For a flat, tabular dataset like CO₂ emissions records, the document model provides no structural advantage and adds query complexity.

Dimension	DuckDB	MongoDB
Data model	Columnar / relational	Document (BSON/JSON)
Query language	SQL	Aggregation pipeline
Analytical queries	Highly optimized	Limited, verbose
Schema	Fixed / typed	Flexible / schemaless
Scaling	Single-node	Horizontal sharding
Best use case	Analytics on tabular data	Flexible document storage

7. Limitations & Lessons Learned

7.1 CSV-to-Parquet Preprocessing Requirement

The original CO₂ emissions dataset was available only in CSV format from Our World in Data. Since a core focus of this project is Parquet-based optimizations — specifically predicate pushdown and projection pushdown — the CSV file had to be converted to Parquet as a preprocessing step before the application could demonstrate these behaviors.

This introduced a dependency that a production system would ideally handle automatically or transparently. In a real analytics pipeline, data ingestion and format conversion would be part of an ETL process rather than a manual step. The requirement to convert the file also means that any updates to the dataset would require re-running the conversion.

This limitation reinforced an important lesson: storage format is a fundamental design decision that must be made early in a data pipeline. The same dataset in CSV versus Parquet produces fundamentally different query execution paths in DuckDB, and the performance implications are significant even at modest data scales.

7.2 Small Dataset Scale

The CO₂ dataset contains 26,508 rows, which is relatively small for a system designed for large-scale analytical workloads. While the performance differences between CSV and Parquet are observable at this scale — with speedups ranging from 3x to 10x — the absolute query times remain in the single-digit millisecond range. This makes it difficult to demonstrate the full magnitude of DuckDB's performance advantages.

At larger scales, such as datasets with millions or tens of millions of rows, predicate pushdown and projection pushdown would yield much more dramatic improvements, as the proportion of skipped data would increase and the I/O savings would dominate overall query time. The benchmark results presented in this project should be interpreted as directionally correct but not representative of DuckDB's full performance potential.

A future iteration of this project could use a larger synthetic dataset or a more comprehensive emissions dataset that includes additional metrics such as total CO₂ output, population, and GDP, which would also enable more complex multi-column analytical queries.

Challenges Faced

One challenge was interpreting DuckDB's ASCII-based EXPLAIN output and mapping each operator back to theoretical database concepts. Another challenge was that query times at this dataset scale are in the millisecond range, making it difficult to produce stable benchmark results without averaging multiple runs.

What Could Be Improved

Future work could use a larger dataset to better demonstrate DuckDB's performance advantages. Additionally, the application could be extended to include write operations, which would allow exploration of DuckDB's behavior under insert and update workloads.

8. Project Links

The project source code, dataset, and documentation are available at the following GitHub repository:

[<https://github.com/JunweiYin2394/dsci551-duckdb-co2>]

The repository contains:

- `app.py` — Streamlit application source code
- `data/` — CO₂ emissions dataset in both CSV and Parquet formats
- The conversion script from CSV to Parquet is included in `duckdb_start.ipynb`
- `README.md` — Setup and run instructions
- `requirements.txt` — Python dependencies